

The self-hosted hackathon operating system

Product Requirements Document and Software Requirements Specification for an offline-first platform that helps participants improve ideas, makes judging defensible, and lets organizers run repeatable events without spreadsheet chaos.

Product promise

From idea intake to published results, every decision is explainable, auditable, portable, and runnable with one command.

Competition strategy

Win on judging integrity and operator usefulness, not on the number of half-finished features.

One specification connects product intent to testable behavior

Purpose

This document is the implementation contract for the planned platform. It combines a Product Requirements Document (PRD) with a Software Requirements Specification (SRS), so product choices, system behavior, acceptance tests, and delivery order stay aligned.

Audience

Product owner, engineering team, designers, test engineers, security reviewers, hackathon organizers, and competition judges.

Field	Value	Owner / decision
Working name	Dogfood Platform	Rename before launch if needed
Version	1.0	Initial planning baseline
Status	Ready for estimation	Open decisions remain on page 20
Primary deployment	Docker Compose	Offline-capable, local dependencies
License target	MIT or Apache-2.0	Final choice before submission
Team size assumption	1-4 builders	Prioritize vertical slices

Interpretation of “rank prediction.” The platform will not promise an exact finishing position before competitors and judges are known. It will provide an explainable **Idea Potential Report**, a rubric-alignment score range, confidence, comparable-pattern signals, and recommended improvements. A simulated rank range may appear only when a real comparison pool exists, with uncertainty shown.

Assumptions used in this version

- The competition brief is the authoritative source for mandatory tiers, offline operation, roles, deliverables, and scoring.
- No external artificial intelligence (AI) service is required at runtime. The MVP can use deterministic rules and a locally packaged model; optional adapters must degrade safely.
- Fixture and historical datasets will be available only when supplied by organizers. The product must not invent “past winner” evidence.
- Email delivery is not required for offline acceptance; invitations are represented by local links and an outbox preview.
- “Event sourcing” applies to audit-critical state changes, not necessarily every database mutation.

How to use this document

Read pages 4-7 to understand the product. Use pages 8-15 to build it. Use pages 16-19 to verify it. Page 20 records decisions still needed.

04 Executive summary

05 Vision, principles, success

06 Users and jobs

07 Lifecycle and scope

08 Participant intelligence

09 Judging intelligence

10 Organizer operations

11 Trust and auditability

12 Core functional requirements

13 Roles and authorization

14 Scoring and ranking

15 Architecture

16 Data model and APIs

17 Non-functional requirements

18 Acceptance and traceability

19 Delivery plan

20 Risks and open decisions

21 Appendix: formulas and states

Requirement language

MUST

Mandatory for correctness, T1/T2, security, or offline operation.

SHOULD

High-value differentiation; implement after mandatory paths are stable.

COULD

Optional enhancement if time remains and it does not endanger core quality.

Scope discipline: A correct T2 platform with one polished differentiator is stronger than a fragile T4 platform. Features are ordered to protect that rule.

The product is an evaluation system, not another submission form

Dogfood Platform is a self-hosted, API-first system for planning, submitting, judging, publishing, and archiving hackathons. Its advantage is a closed loop: participants receive honest guidance before submission; judges work faster with calibrated, evidence-led tools; organizers get defensible rankings and repeatable operations.

01

Build better

Idea Potential Reports expose rubric gaps, team-skill mismatch, unrealistic scope, and missing evidence before submission.

02

Judge fairly

Anchor calibration, documented normalization, disagreement routing, and weight sensitivity make scoring explainable.

03

Run repeatedly

Event cloning, judge passports, reusable policies, exports, and queryable archives reduce work across events.

The key product bet

Competitors are likely to reproduce the familiar event microsite, registration, forms, gallery, and dashboard. This product treats those as the reliable foundation. Differentiation comes from the decisions most platforms leave to spreadsheets or human memory: whether an idea fits the rubric, whether a judge is systematically harsh, where reviewers disagree, how weights alter rankings, who changed what after a deadline, and how the next event reuses what the organization learned.

Primary outcome

An organizer can create an event, accept a project, assign independent judges, compute a transparent ranking, publish results, and export the full record on a laptop with the network disabled.

North-star metric

Median organizer minutes per completed project lifecycle, measured from registration through archived result, while correctness and auditability gates remain green.

Definition of success

- ✓ T1 and T2 acceptance suites pass.
- ✓ Every rank can be traced to source ballots and formulas.
- ✓ Another operator launches the seeded portal from written instructions.
- ✓ Participant guidance explains uncertainty and never fabricates comparisons.
- ✓ All event data can leave through documented exports.

Trust, speed, and portability govern every product decision

Vision

Any community should be able to run a credible technical competition without surrendering data to a hosted vendor or manually repairing the judging process.

Mission for the hackathon release

Deliver the smallest complete product that proves an event can move from idea to archived result with reliable role isolation, weighted judging, documented normalization, and one memorable participant-facing intelligence feature.

Product loop

Coach ideas



Collect proof

Judge
independentlyResolve
uncertainty

Publish + learn



Clone next event

Product principles

Principle	Product consequence
Explain before automate	Every score adjustment shows inputs, method, confidence, and effect.
Offline is a feature	Core workflows require no external service, account, or API key.
Backend owns policy	Authorization, deadlines, track isolation, and finalization are server-enforced.
Uncertainty is data	Conflicting reviews trigger action instead of being hidden by an average.
Portability prevents lock-in	Imports, exports, OpenAPI, and webhooks cover the usable product surface.
Operators need memory	Reusable event templates and judge performance records work across events.

Success measures for the first usable release

Setup

Seeded portal usable within 10 minutes of a documented local start.

Judge speed

A trained judge completes a rubric without a mouse and without seeing peer scores.

Traceability

100% of published results resolve to ballots, rubric version, and ranking run.

Five roles share one lifecycle but must never share unauthorized state

Participant

Job: form a team, test an idea against the event rubric, submit convincing evidence, and understand status.

Pain: vague criteria, oversized scope, missing evidence, late surprises.

Judge

Job: review assigned projects quickly, independently, consistently, and with confidence.

Pain: context switching, unclear rubrics, fatigue, calibration drift.

Organizer

Job: configure policy, monitor progress, resolve exceptions, publish defensible results, and reuse the setup.

Pain: spreadsheets, reminders, opaque normalization, missing audit trails.

Administrator

Job: operate the instance, manage global policy, recover safely, and export all data.

Pain: vendor lock-in, hidden dependencies, weak backups, privilege ambiguity.

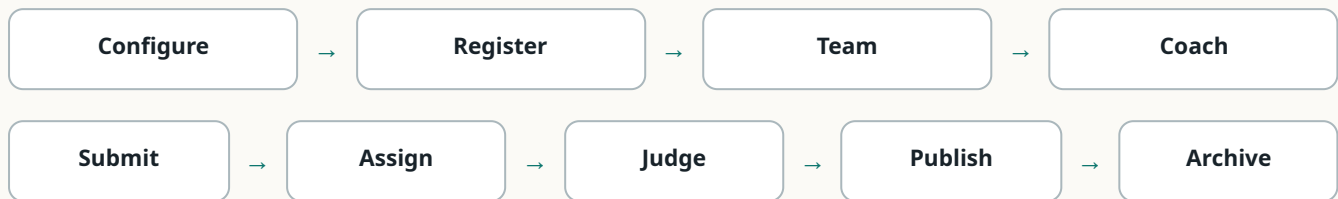
Visitor / community voter

Job: discover eligible projects and, where enabled, vote or comment without compromising the competition. Public access must be configurable and results remain hidden during the voting window.

Critical jobs-to-be-done

1. When I have an idea, help me see how it aligns with this event's actual rubric and what I should change before committing.
2. When I judge many projects, keep me in flow and stop other judges' opinions from influencing my ballot.
3. When reviewers disagree, direct scarce judging capacity to the projects where one more review changes confidence most.
4. When I publish winners, let me explain exactly how the result was produced and what changed after deadlines.
5. When I run another event, let me reuse the proven configuration without copying spreadsheets.

The release covers one complete path before it expands sideways



Release scope by priority

Priority	Included capability	Exit condition
P0 / Foundation	Auth, roles, events, teams, submissions, deadlines, gallery, seed, Docker	T1 acceptance passes
P0 / Judging	Rubrics, assignments, private ballots, progress, weighted ranking, exports	T2 acceptance passes
P1 / Signature	Idea Potential Report, calibration anchors, disagreement routing, weight simulator	One end-to-end story polished
P1 / Trust	Version freeze, audit timeline, attack tests, finalization lock	Critical abuse tests pass
P2 / Operations	Event clone, Judge Passport, archive intelligence, public voting	Does not destabilize P0/P1
P3 / Extension	Pairwise tie-break, certificates, signed participation record, widget	Only after all gates

Explicitly out of scope for MVP

- ✗ Hosted-only dependencies
- ✗ Exact rank promises from incomplete data
- ✗ Autonomous AI judging
- ✗ Native mobile applications
- ✗ Payments or prize disbursement
- ✗ Real-time video hosting

Guardrails

- ✓ No participant data trains a model by default.
- ✓ No peer ballot is exposed to a judge.
- ✓ No post-deadline edit replaces frozen content.
- ✓ No algorithmic output lacks an explanation path.
- ✓ No export omits identifiers needed to reconstruct state.

The Idea Potential Report is a coach, not a fortune teller

The participant enters an idea, selected track, proposed features, expected stack, available hours, and optional team skills. The system maps that input to the event's published rubric and returns evidence-backed ranges, risks, and next actions.

Required outputs

- **Rubric alignment:** a score band per criterion, not a false precise score.
- **Confidence:** high, medium, or low with the reason for uncertainty.
- **Scope pressure:** likely achievable, at risk, or unrealistic for the declared team-hours.
- **Team fit:** capabilities required, capabilities present, and specific gaps.
- **Blind spots:** missing architecture, validation, demo, security, or evidence.
- **Improvement plan:** the three changes most likely to improve rubric alignment.
- **Comparable patterns:** only from organizer-provided, provenance-linked history.

Example output contract

Overall potential: 68-78 / 100
Confidence: Medium

Technical depth: 12-16 / 20
Reason: architecture named; failure mode absent

Feasibility: At risk
Reason: 10 features / 48 team-hours

Best next action:
Cut real-time chat and prove judge isolation.

Illustrative format only. Production values must be computed from the event rubric and submitted inputs.

Prediction modes

Mode	When allowed	What is shown
Idea-only	No comparison pool	Rubric bands + recommendations
Cohort simulation	Enough real submissions exist	Percentile / rank interval + confidence
Final projection	Organizer enables it	Scenario range, never an official result

Non-negotiable: The tool must say what data it used, what it did not know, and that it cannot predict judge preference or unseen competitors. It may not display "you will rank #3."

Functional requirements

AI-01 The system MUST accept idea text and event context without publishing the idea. **AI-02** It MUST generate criterion-level explanations. **AI-03** It MUST suppress historical claims when no grounded dataset exists. **AI-04** Participants MUST be able to delete reports. **AI-05** Re-running after edits MUST show what changed and why.

Calibration and disagreement make scarce reviews more valuable

Judge calibration protocol

Before live assignments, a judge scores three organizer-approved anchor projects spanning weak, typical, and strong performance. The platform estimates severity and criterion drift, then shows private guidance such as “your innovation scores trend 0.7 below the panel anchor.”

Calibration affects normalization only when the organizer enables a documented method.

Disagreement-driven reviews

After the minimum review count, the platform computes uncertainty by score dispersion, criterion disagreement, and reviewer reliability. Projects above the threshold receive another eligible judge; stable projects stop consuming reviews.

Assignments remain track-safe and conflict-aware.

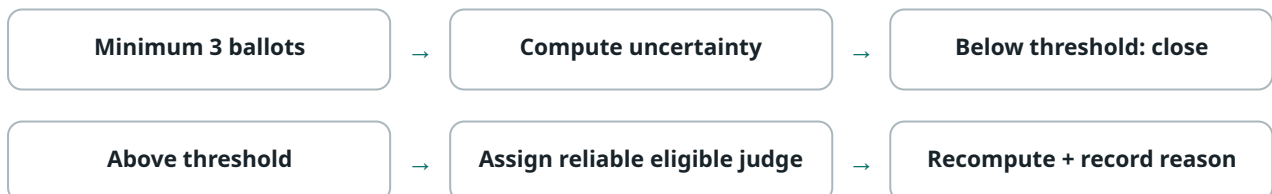
Weight sensitivity simulator

Organizers adjust criterion weights in a sandbox and see rank movement before locking the rubric or publishing results. The tool highlights fragile ranks: projects whose position changes materially under small reasonable weight variations.

Hybrid pairwise tie-break

For a small cluster within an organizer-defined score margin, judges may compare two projects directly. Pairwise results supplement, rather than silently replace, weighted rubric scores.

Adaptive review policy



Judging requirements

JDG-01 The system MUST preserve raw scores. **JDG-02** Normalized scores MUST never overwrite ballots.

JDG-03 Every ranking run MUST store method, parameters, rubric version, input ballot IDs, and output order.

JDG-04 Extra assignments MUST record the trigger. **JDG-05** A judge MUST NOT see aggregate or peer scores before publication.

Cross-event memory turns a weekend tool into operating infrastructure

Judge Passport

A private, organizer-visible record carries forward a judge's eligible tracks, conflicts, assignment completion, median review time, calibration behavior, and reliability. It does not create a public reputation score.

- Separate observed metrics from organizer notes.
- Expire or recalculate metrics as rubrics change.
- Allow judge data export and deletion.
- Never infer protected attributes.

Keyboard-first judge console

J/K moves between projects; number keys set a score; E jumps to evidence; F flags; S saves. Every shortcut has a visible button and accessible label. Autosave state is explicit.

One-click event clone

Clone tracks, rubric structure, policies, custom questions, templates, and eligible judge pool into a draft. Dates, submissions, ballots, votes, and private participant data never copy.

- Show a pre-clone checklist.
- Version copied rubrics.
- Require confirmation before invitations.
- Record the source event ID.

Archive intelligence

Filter historical events by track, date, technology, placement, and score range. Natural-language search is optional; every answer must link back to actual archived records.

Organizer command center

Question answered	Signal	Action
Who has not started?	Assignment status	Reassign or remind
Which projects are uncertain?	Dispersion + confidence	Add targeted review
Is a track overloaded?	Queue / capacity ratio	Move eligible judges
Will weights destabilize ranks?	Sensitivity analysis	Review rubric before lock
What changed after freeze?	Version timeline	Accept exception or reject

OPS-01 The dashboard MUST show status without exposing hidden ballots to unauthorized roles. **OPS-02** Bulk actions MUST provide a preview and confirmation. **OPS-03** Cloning MUST produce a draft event. **OPS-04** Historical analytics MUST distinguish observed facts from derived metrics.

The system proves policy with behavior, history, and attack tests

Submission lineage and freeze

Every material submission save creates a version containing content, editor, timestamp, and parent version. At the deadline, the accepted version is frozen and hashed. Later amendments are separate versions with explicit organizer approval and reason.

Time-travel audit

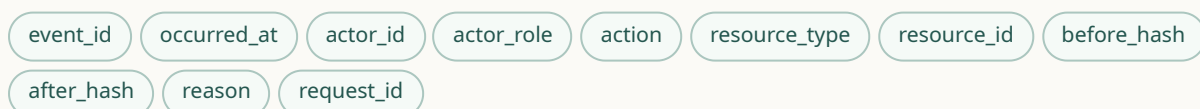
An organizer can reconstruct the state at a selected timestamp: rubric version, assignments, ballots submitted, eligibility status, ranking run, and publication state. The interface never rewrites history.

Executable threat model

Attack	Expected control	Test result
Read peer ballot	Backend policy	403 + audit event
Late submission	Server deadline	422 + preserved draft
Ballot stuffing	Rate + identity rules	429 / flagged cluster
Edit finalized result	State lock	409 + audit event
Track crossover	Assignment scope	403
Audit deletion	Append-only policy	Denied

Hashing is evidence, not magic. A local hash chain detects modifications inside the exported history. It does not prove who controlled the host. Documentation must state this boundary and avoid blockchain claims.

Audit event minimum fields



TRU-01 Audit records MUST be append-only through application APIs. **TRU-02** Sensitive payloads MUST be redacted. **TRU-03** Deadline checks MUST use server time. **TRU-04** Every override MUST require a reason. **TRU-05** The attack suite MUST run locally.

The mandatory flow is complete, backend-enforced, and testable

ID	Requirement	Acceptance evidence
AUTH-01	Local account registration, login, logout, and session expiry	Valid / invalid session tests
AUTH-02	Roles: visitor, participant, judge, organizer, admin	API authorization matrix
EVT-01	Create and edit event dates, tracks, prizes, visibility, and state	Persisted configuration
EVT-02	Configure rubric criteria, scales, weights, guidance, and versions	Weights validate and lock
TEAM-01	Create team and join through expiring invite link	Single membership policy
SUB-01	Create draft with standard and organizer-defined fields	Draft persists after restart
SUB-02	Edit until server-enforced deadline; freeze accepted version	Boundary-time tests
GAL-01	Public gallery supports search, track filter, tags, and eligibility	Query and visibility tests
ASN-01	Batch or algorithmic assignment honors capacity, track, conflict	No invalid assignment
BAL-01	Judge scores weighted criteria and writes feedback	Rubric validation
BAL-02	Draft and submit ballot; submitted ballots follow lock policy	State transition tests
RES-01	Compute rankings from eligible submitted ballots	Fixture expected outputs
RES-02	Preview, finalize, and publish results as distinct states	Unauthorized access denied
EXP-01	CSV and JSON export at registration, submission, judging, results	Round-trip sample
ARC-01	Archive an event without deleting queryable records	Archive search test

Submission state machine

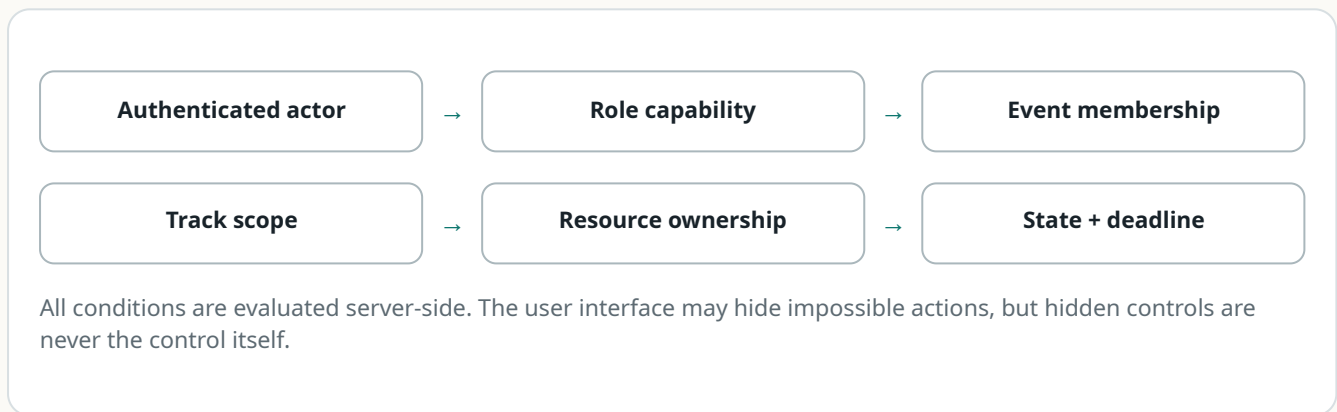


A post-freeze correction creates an amendment request. It never mutates the frozen version. The organizer may approve, reject, or annotate the request.

Permissions are evaluated at the resource, event, and track levels

Actor	Own data	Peer ballots	Event control	Audit
Visitor	Public only	Never	None	None
Participant	Team + submission	Never	None	Own history
Judge	Assignments + ballots	Never pre-publish	None	Own actions
Organizer	Event scoped	Authorized	Own events	Event scoped
Admin	Instance scoped	Break-glass only	All events	Instance scoped

Authorization decision



Security requirements

- **SEC-01** Passwords MUST use a memory-hard hash with per-user salts.
- **SEC-02** Session cookies MUST be HttpOnly, SameSite, and Secure when TLS is enabled.
- **SEC-03** State-changing requests MUST resist cross-site request forgery.
- **SEC-04** User content MUST be escaped or sanitized to prevent script execution.
- **SEC-05** Rate limits MUST apply to login, voting, comments, invitations, and report generation.
- **SEC-06** Admin elevation and break-glass access MUST create high-severity audit events.
- **SEC-07** Secrets MUST arrive through runtime configuration, never the repository or seed data.

Privacy rule: Judge Passport metrics are operational records, not a public leaderboard. Access is limited, retention is configurable, and raw comments must not leak into exports intended for participants.

Raw evidence is immutable; derived rankings are versioned calculations

Base score

```
project_score =  $\sum$  (criterion_score  $\times$  criterion_weight)
where  $\sum$  criterion_weight = 1.0
```

Each ballot retains its raw criterion values. The event selects one documented aggregation method. The default is the mean of complete eligible ballots, with minimum review count enforced.

Proposed normalized method

1. For each judge and criterion, estimate location and scale from anchor and live ballots when sample size is sufficient.
2. Convert raw criterion scores to standardized values with bounded scale floors.
3. Map standardized values back to the rubric scale for interpretation.
4. Aggregate across judges using the locked rubric weights.
5. Publish raw and normalized views to organizers, with rank deltas.

Fallback hierarchy

1. Anchor-informed adjustment
2. Robust per-judge standardization
3. Raw weighted mean

The method chosen for an event is locked before publication. Small samples use the safer fallback; the UI explains why.

Uncertainty score

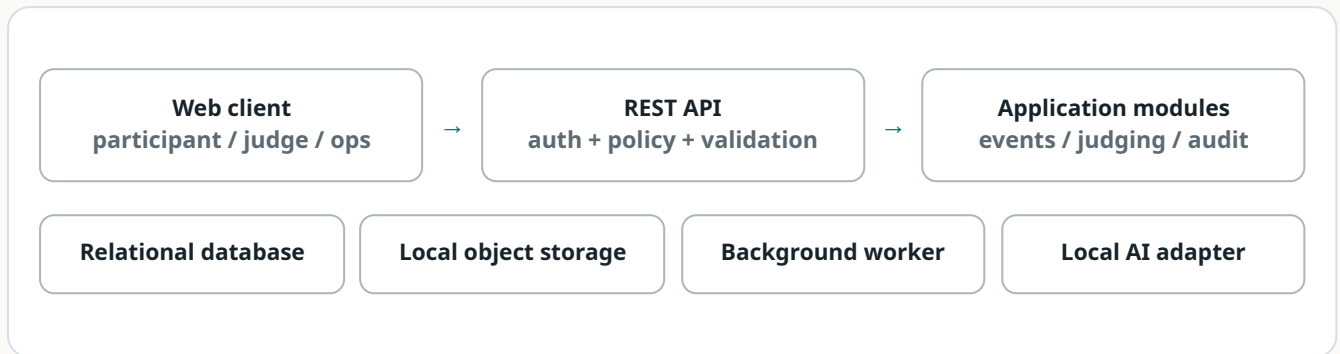
A project is flagged when any configured condition holds: weighted-score standard deviation above threshold; criterion spread above threshold; one ballot is an outlier under the documented rule; rank confidence interval overlaps a prize boundary; or missing review count threatens completion.

Ranking rules

Case	Rule
Missing ballot	Exclude draft; show completeness warning
Ineligible project	Exclude from publishable ranking
Exact tie	Organizer-defined tie-break; never random silently
Weight change	Create new ranking run; retain prior run
Late override	Reason + actor + timestamp + rerun required

Ranking simulation: A participant-facing interval may be computed only from a real, eligible cohort and a declared model. It must be labeled a simulation and must not reveal competitors' private data.

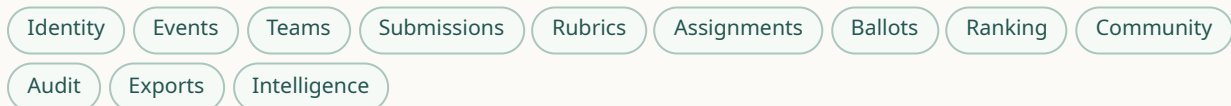
A modular monolith minimizes delivery risk without closing the API



Recommended deployment shape

Service	Responsibility	Offline default
app	Server-rendered UI or SPA + API	Required
db	Relational records and indexes	Required
worker	Exports, reports, certificates	May share app process
object store	Uploaded media and export bundles	Filesystem or local service
model	Idea report inference	Rules / packaged local model

Module boundaries



Why this shape

- One deployable unit fits the 72-hour constraint.
- Module boundaries still map cleanly to API resources.
- Database transactions protect cross-module invariants.
- Future services can split at stable boundaries.

Failure behavior

- Core submission and judging continue if intelligence is unavailable.
- Queued exports retry with bounded attempts.
- Ranking failures leave previous runs untouched.
- Startup health checks report missing migrations or storage.

Stable identifiers and versioned records make the event reconstructable

Core entities

Domain	Entities	Critical invariant
Identity	User, Session, Membership	Role is event-scoped where applicable
Event	Event, Track, Prize, Question	Dates use explicit timezone / UTC storage
Team	Team, TeamMember, Invite	Membership rules are event-specific
Submission	Project, Version, Asset, Eligibility	Frozen version is immutable
Judging	RubricVersion, Assignment, Ballot, Score	Ballot references exact rubric version
Results	RankingRun, RankedProject, Publication	Inputs and method are retained
Trust	AuditEvent, Override, Conflict	Application path is append-only
Intelligence	IdeaReport, FeatureVector, CohortRun	Input provenance + model version retained

API conventions

- Versioned base path, resource-oriented JSON, consistent pagination, filtering, and error envelopes.
- Idempotency keys for assignment generation, result finalization, bulk import, and webhook retries.
- Optimistic concurrency for event configuration and submission versions.
- OpenAPI document generated and committed; every UI mutation has an equivalent API operation.
- Webhooks for submission frozen, assignment created, ballot submitted, results published, and event archived.

```

POST /api/v1/events/{eventId}/idea-reports
POST /api/v1/events/{eventId}/assignments:generate
POST /api/v1/events/{eventId}/ranking-runs
POST /api/v1/events/{eventId}/results:publish
GET /api/v1/events/{eventId}/exports/{stage}

```

Export contract

Each export bundle includes schema version, generated timestamp, event ID, data files, asset manifest, checksums, and a human-readable README. A full export must be sufficient to reconstruct the event in a clean instance after compatible import.

Adoptability is measurable behavior, not a README adjective

ID	Target	Verification
NFR-01	Seeded portal starts with one documented command	Clean-machine run
NFR-02	Core flows work with network disabled	Offline acceptance run
NFR-03	95th percentile API read under 500 ms for fixture scale	Local load test
NFR-04	Submission save is durable after acknowledged response	Restart test
NFR-05	No critical authorization bypass in attack suite	Automated negative tests
NFR-06	WCAG 2.2 AA for critical flows	Keyboard + automated audit
NFR-07	All timestamps stored in UTC; event timezone displayed	Boundary and DST tests
NFR-08	Full backup and documented restore	Restore into clean volume
NFR-09	Structured logs exclude secrets and private ballot content	Log inspection tests
NFR-10	Migrations are ordered, reversible when practical, and backed up	Upgrade rehearsal

Accessibility

- Complete keyboard operation
- Visible focus and skip links
- Labels and errors tied to fields
- No color-only meaning
- Reduced-motion support

Compatibility

Latest stable Chrome, Firefox, and Safari; responsive layouts down to 360 px. Administrative bulk workflows may recommend desktop but must remain usable.

Observability

- Health and readiness endpoints
- Request IDs through audit events
- Metrics for queues, errors, latency
- Ranking run diagnostics
- Storage and backup warnings

Retention

Configurable retention by data class. Deletion requests remove personal data where lawful while preserving minimally necessary integrity records through pseudonymization.

Fixture-scale baseline: tests should cover at least 40 projects, 30 judges, three reviews per project, concurrent judging, and representative media. Larger-scale claims require separate evidence.

Every differentiator has a concrete pass condition

Capability	Given / when	Pass condition
Idea report	Idea + rubric submitted	Bands, reasons, confidence, actions
No fake history	No comparison dataset	No historical or rank claim appears
Calibration	Judge scores anchors	Private bias view; raw scores unchanged
Disagreement	Ballots exceed threshold	Eligible fourth review proposed
Weight simulator	Organizer adjusts weights	Rank deltas shown; no saved result mutates
Judge isolation	Judge requests peer ballot	403 and auditable denial
Deadline	Save after close	Frozen version unchanged
Time travel	Organizer selects timestamp	Reconstructed state matches events
Clone	Prior event cloned	Policy copied; private records absent
Export	Full bundle imported cleanly	Counts and hashes reconcile

Definition of done for a requirement

- ✓ Behavior implemented through the backend, not only the interface.
- ✓ Happy path, validation error, authorization failure, and state boundary are tested.
- ✓ Audit event exists where the action changes competition state.
- ✓ API and interface use the same domain rule.
- ✓ Accessibility labels and keyboard path are verified.
- ✓ Documentation states known limits without inflated tier claims.

Release gates

G1

Correctness

T1/T2 suite and project tests pass.

G2

Operability

Fresh offline startup and restore pass.

G3

Integrity

Attack suite and ranking fixtures pass.

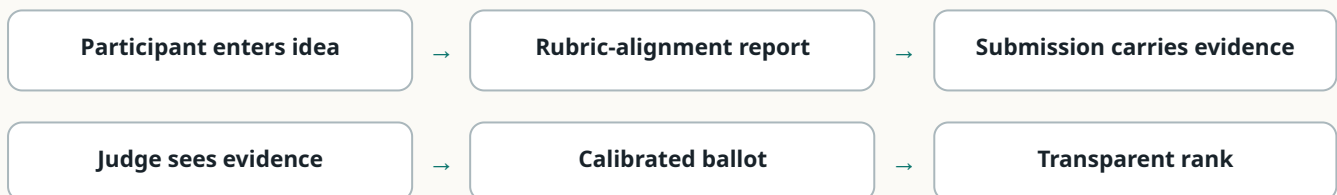
Stop rule: If a P1 feature threatens any gate, disable it cleanly behind configuration and ship the stable core. "Present but unreliable" is not completion.

Build vertical slices in the order that protects the mandatory tiers

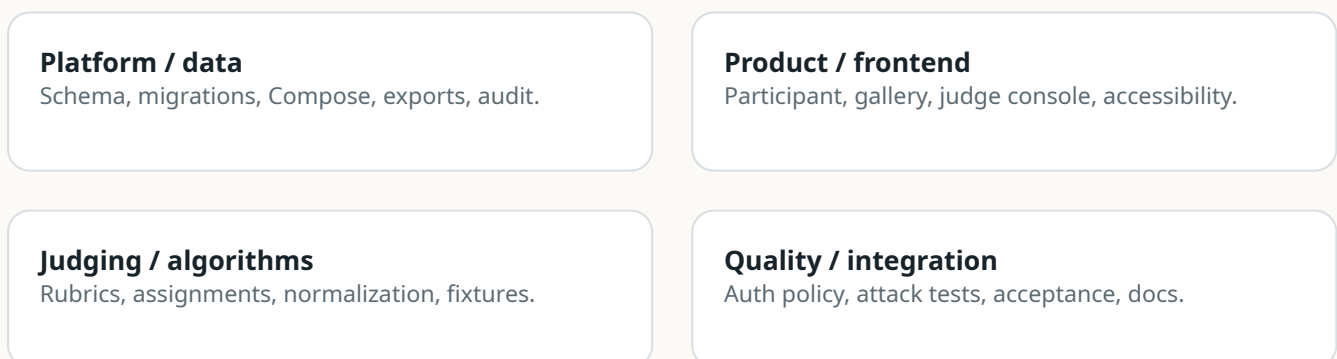
72-hour implementation sequence

Window	Primary output	Exit check
0-6 hours	Schema, state machines, repo, Compose, seed	Fresh start works
6-18 hours	Auth, roles, events, teams	Policy tests pass
18-30 hours	Submission versions, deadline, gallery	T1 path complete
30-44 hours	Rubrics, assignment, judge console	Private ballot path complete
44-54 hours	Ranking, export, progress dashboard	T2 path complete
54-62 hours	One signature vertical slice	Idea report or calibration polished
62-68 hours	Threat tests, audit, docs	Integrity gates pass
68-72 hours	Acceptance, fixes, final artifacts	No new scope

Recommended signature slice



Ownership model for a four-person team



For smaller teams, merge roles but retain the sequence. Do not parallelize dependent foundations before contracts are agreed.

The biggest risks are false confidence, scope spread, and hidden policy

Risk	Impact	Mitigation
Idea report feels like fake AI	Trust loss	Ranges, explanations, grounded inputs
Normalization overfits small samples	Ranking distortion	Fallback hierarchy + raw comparison
Too many unique features	Broken T1/T2	One signature slice; feature flags
Audit log stores sensitive content	Privacy breach	Metadata + hashes; explicit redaction
Judge Passport becomes reputation score	Unfair treatment	Private metrics, context, expiry, review
Adaptive assignment creates bias	Uneven scrutiny	Document thresholds and review caps
Offline model too heavy	Startup failure	Rules baseline; optional local model
Event clone copies private data	Data leakage	Allowlist configuration fields only

Decisions required before implementation

1. **Stack:** choose web framework, relational database, job runner, and local object storage.
2. **Idea engine:** deterministic rubric rules only, packaged local model, or both.
3. **Normalization:** anchor-adjusted, robust standardization, or raw mean as default.
4. **Voting:** include community voting in MVP or leave behind a feature flag.
5. **Judge Passport retention:** define duration, deletion, and cross-event consent.
6. **Uploads:** local size limits, allowed types, virus scanning strategy, and demo constraints.
7. **Licensing:** MIT or Apache-2.0.

Recommended decisions

Ship: PostgreSQL or SQLite-compatible relational design, modular monolith, rules-first Idea Potential Report, raw weighted mean plus documented optional normalization, local filesystem assets, OpenAPI.

Defer: open public voting, full pairwise ranking, certificate signing, embeddable widget, and natural-language archive search until the foundation is verified.

State machines, formulas, and submission artifacts

Event states

```
DRAFT
-> REGISTRATION_OPEN
-> SUBMISSION_OPEN
-> SUBMISSION_FROZEN
-> JUDGING_OPEN
-> RESULTS_FINALIZED
-> RESULTS_PUBLISHED
-> ARCHIVED
```

Ballot states

```
UNASSIGNED
-> ASSIGNED
-> IN_PROGRESS
-> SUBMITTED
-> LOCKED
-> REOPENED (override)
-> SUBMITTED
```

Key calculations

```
weighted_ballot = sum(score[c] * weight[c])
raw_project = mean(weighted_ballot[j])
uncertainty = f(score_spread, criterion_spread, review_count,
                reviewer_reliability, prize_boundary_overlap)
rank_delta = simulated_rank - baseline_rank
```

Required repository artifacts

File	Purpose
README.md	Run, use, limitations, tier claim
ARCHITECTURE.md	System shape and trade-offs
DATA-MODEL.md	Schema, versioning, import / export
JUDGING.md	Assignment, scoring, normalization
THREAT-MODEL.md	Assets, actors, attacks, controls, gaps
openapi.yaml	Public API contract
acceptance-report.txt	Verified tier receipt
.dogfood.toml	Honest tier claim and pitch

Glossary

Anchor project: a reference submission used to calibrate judges. **Ballot:** one judge's scores and feedback for one project. **Normalization:** a documented adjustment for systematic judge scale differences.

Ranking run: immutable calculation record producing an ordered result. **Rubric version:** locked criteria, scales, weights, and guidance. **Uncertainty:** evidence that another review may materially change confidence.

Product sentence: Dogfood Platform helps teams submit stronger projects, helps judges reach fairer decisions faster, and gives organizers a complete record they can run again.